

Academic Staff Recruitment System using React.js, Node.js, MongoDB, and API Integrations

FRANCKY RONSARD SAAH
TECHNOLOGY FACULTY
INFORMATION SYSTEM
ENGINEERING
KOCAELI, TURKEY
francky877832@gmail.com

KHADIM DIEYE
TECHNOLOGY FACULTY
INFORMATION SYSTEM
ENGINEERING
KOCAELI, TURKEY
Khadimd978@gmail.com

TURAN ASGARLI
TECHNOLOGY FACULTY
INFORMATION SYSTEM
ENGINEERING
KOCAELI, TURKEY
turanaskerov2004@gmail.com

Abstract- *The recruitment process for academic staff is a critical task that requires an efficient and reliable system to handle applications, evaluations, and approvals. This project presents a web-based academic staff recruitment system that streamlines the application process using modern web technologies. The system is developed using React.js for the frontend, Node.js (Express.js) for the backend, and MongoDB as the database. Additionally, it integrates third-party services such as Amazon S3 for file storage, Twilio for notifications, and JWT-based authentication for security.*

A key feature of this system is its integration with governmental APIs (e-Devlet & Nüfus Müdürlüğü) to validate applicant credentials and streamline the verification process. The system provides multiple user roles, including applicants, administrators, managers, and jury members, each with specific functionalities to manage applications, evaluate candidates, and finalize recruitment decisions.

This paper details the architecture, implementation, and functionalities of the system. The results indicate that the platform significantly reduces administrative overhead, improves application tracking, and enhances the efficiency of the recruitment process. Future improvements may include AI-driven candidate ranking and enhanced security mechanisms.

Keywords: Academic Recruitment, React.js, Node.js, MongoDB, API Integration, Authentication, AWS S3, Twilio, Brevo

I. INTRODUCTION

1. Background

The recruitment of academic staff plays a vital role in ensuring the quality of education and research in universities. Traditionally, this process involves **manual submission of applications, document verification, and evaluation by selection committees**, leading to inefficiencies such as delays, mismanagement of documents, and lack of transparency. With the growing need for **digital transformation in academia**, there is a demand for an automated system that streamlines this process while ensuring security, scalability, and integration with official government databases.

2. Objectives

The primary objective of this project is to **develop a web-based academic recruitment system** that allows institutions to manage applications digitally, ensuring:

- **Seamless application submission** for academic positions.
- **Automated verification** of applicants' credentials using **e-Devlet & Nüfus Müdürlüğü API**.
- **Secure document storage** using **Amazon S3**.
- **Multi-role system** with distinct functionalities for applicants, administrators, managers, and jury members.

Notification and communication system powered by **Twilio**.

- **Authentication and access control** using **JWT**.

3. Problem Statement

Existing recruitment processes in many academic institutions face several challenges:

- **Time-consuming and inefficient workflows** due to manual handling of applications.
- **Document mismanagement**, leading to lost or misplaced credentials.
- **Lack of real-time updates**, requiring candidates to frequently inquire about their application status.
- **Limited integration with external verification systems**, making it difficult to validate applicant credentials efficiently.
- **Security risks** in handling sensitive personal and academic data.

This project addresses these issues by implementing a **modern, secure, and scalable recruitment platform** that enhances the efficiency of the recruitment process.

4. Contributions

This system contributes to the academic recruitment process by:

- **Automating the entire recruitment workflow**, reducing administrative overhead.
- **Providing a centralized platform** for applicants, administrators, and evaluators.

- **Ensuring security and integrity** of applicant data through authentication and secure storage.
- **Enhancing transparency** by providing real-time application status updates.
- **Facilitating document verification** via API integration with government systems.

5. Organization of the Paper

The remainder of this paper is structured as follows:

- **Section 3** describes the system architecture and the technologies used.
- **Section 4** presents the system components, detailing the frontend, backend, database, authentication, and third-party integrations.
- **Section 5** discusses the system’s functionalities and user roles.
- **Section 6** provides insights into implementation details, including key code segments.
- **Section 7** presents results, testing, and performance analysis.
- **Section 8** concludes the paper and outlines future work.

II. SYSTEM ARCHITECTURE

1. Overview

The Academic Staff Recruitment System is designed as a web-based platform that follows a modern, scalable, and secure architecture. It is developed using a client-server model, where the frontend (**React.js**) interacts with the backend (**Node.js/Express**) via a RESTful API. The backend communicates with a **MongoDB database (noSQL)** for data persistence and integrates with external services such as **Amazon S3 for file storage, Twilio for notifications, and e-Devlet & Nüfus Müdürlüğü APIs for identity verification.**

The system is built with the following architectural considerations:

- **Scalability:** The modular architecture allows the system to scale efficiently.
- **Security:** Implements authentication, authorization, and secure API communications.
- **Integration:** Supports external API connections for validation and notifications.
- **User Role Management:** Handles multiple user types with different access levels.

2. Technologies Used

The system relies on a full-stack JavaScript ecosystem, with the following key technologies:

	Technology Used	Purpose
Frontend	React.js, Bootstrap	User interface design

Backend	Node.js, Express.js	API development and business logic
Database	MongoDB	Data storage and management
Authentication	JSON Web Token (JWT)	Secure user authentication
File Storage	Amazon S3	Secure document upload and retrieval
Notifications	Twilio API,	SMS/Email alerts for users
	Brevo API, Built-in Notification	Email alerts for users
API Integration	e-Devlet & Nüfus Müdürlüğü	Identity and document verification

3. System Architecture Diagram

The system follows a three-tier architecture, which consists of:

- **Presentation Layer (Frontend):** React.js handles the user interface, enabling users to interact with the system.
- **Business Logic Layer (Backend):** Node.js processes requests, implements authentication, and connects with external APIs.
- **Data Layer (Database & External Services):** MongoDB stores user and application data, while external APIs handle verification(E-devlet), file storage (Amazon S3) and notifications(Twilio, Brevo).

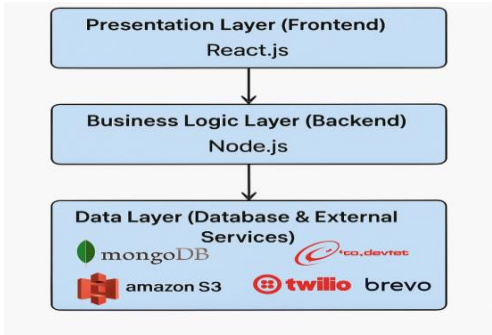


Fig 1.

4. Interaction Flow

The system operates as follows:

A. Frontend (React.js & Bootstrap) - User Control, Registration & Authentication

Phase 1: Token Validation and User Redirection

- **Token Check:** When a user accesses the system, the application first checks for the presence of a valid JWT token in the browser’s local storage.

- **Automatic Login:** If a valid token is found, the user is automatically logged in, and a new JWT token is generated with a validity period of 7 more days. This process ensures that users don't need to log in every time they visit the system within the token's expiration period.
- **No Token Found:** If no valid token exists, or if it's the user's first visit to the system, the user is redirected to the **ControlUser** page to begin the registration process.

Phase 2: User Data Entry and e-Devlet API Verification

- **Personal Information Submission:** On the **ControlUser** page, users are required to enter essential personal details, such as **TC Kimlik**, **Ad**, **Soyad**, and **Doğum Yılı**.
- **SOAP API Call:** The entered data is sent to the backend, which then uses the **SOAP protocol** to query the **e-Devlet API** for user verification. This step ensures that the candidate exists in the government database.
- **Boolean Response:** The backend returns a boolean indicating whether the user exists. If the user is not found, an error message is displayed to the user.
- **Verified User Redirection:** If the user is found in the e-Devlet system, they are redirected to the login page to complete their registration.

Phase 3: User Registration and Authentication

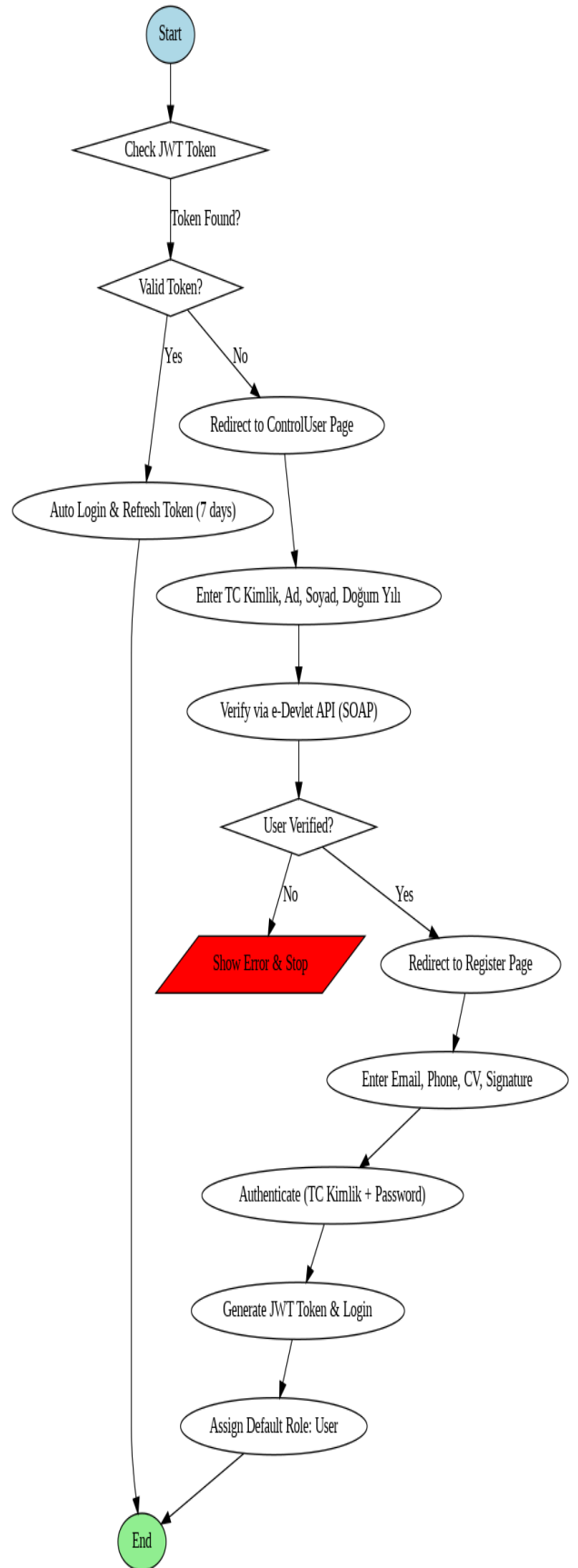
- **Completing Registration:** On the login page, users are prompted to provide additional registration details, including their **email**, **phone number**, **CV**, and a **signature image** to be used on their application form.
- **Authentication Process:** Once the user submits this information, they are authenticated using their **TC Kimlik** and a password. At this point, a new JWT token is generated, and the user is logged in automatically.

Phase 4: Role Assignment and Access Control

- **Default User Role:** Initially, all new users are assigned the default role of **user**.

Note : Role Management: Only the system **manager** has the authority to modify user roles, allowing them to upgrade users to roles such as **administrator** or **manager** based on the requirements. This role-based access control ensures that users have appropriate permissions within the system.

Here can be the flow chart of the above process :



B. Backend (Node.js & Express.js) - Application Submission

The application submission process is a critical component of the system, designed to streamline the application for academic positions by ensuring all candidate details are entered accurately and securely. It also ensures that the necessary documents are submitted and stored for later evaluation by the jury.

Phase 1: Accessing Job Announcements and Login/Registration

Homepage: The homepage presents the latest academic job announcements, categorized by position type: Assistant Professor, Associate Professor, Professor, and All.

Login/Registration: If the user is not logged in, they are redirected to the login or registration page. After successful login, the full job announcement is displayed. The user can then click the "Apply" button to proceed.

Phase 2: Dynamic Multi-Step Application Form

- **Application Form Generation:** After clicking the "Apply" button, the user is directed to a multi-step application form. The form is dynamically generated from the MongoDB "forms" collection, where each form is stored as a document. Each form corresponds to the categories of the KOÜ Atama Yo netmeligi Table 5.

- **Form Structure:** Each form consists of 12 steps representing different categories in Table 5 of KOÜ Atama Yonetmeligi.

Each step includes a heading (H1) indicating the category (e.g., **A Makaler**), followed by a dropdown menu for selecting the appropriate activity within that category (e.g., 1 to 8 for Activity A).

- Filling Out the Form:

Once an activity is selected, a blank form appears, asking the candidate to fill in the necessary information and upload proof documents for that specific activity. These documents will be used by the jury to verify the validity of the information provided.

Candidates can fill out as many activities as needed from different categories but can only submit one activity per category at a time.

- Navigation Between Steps:

After completing a step, the candidate can click on "Next" or "Previous" to navigate through the form. The form allows flexibility, enabling users to fill in as many activities as necessary.

Phase 3: Review and PDF Generation

- **Review Page:** After completing the form, the user is directed to the "Review" page.

A PDF document is automatically generated, which includes all the completed activities, the points calculated according to KOÜ Atama Yo netmeligi Table 4, and the candidate's signature (from the registration process).

The candidate is also presented with four radio buttons to choose the evaluation basis for the jury (**Puanlanan Faaliyet Dönemi**).

- PDF Customization:

The generated PDF contains the activities filled out by the candidate, their calculated points, and the candidate's signature. The system also includes the option for the user to review and modify any data in the form before finalizing the application.

Phase 4: Submission and Application Tracking

- Finalizing the Application:

Once the candidate reviews the details in the PDF and is satisfied with the information provided, they click the "Generate PDF" button to create the final document.

After the PDF is generated, the candidate clicks the "Submit Application" button to submit their application for review by the jury.

- Tracking Application Status:

The candidate is then redirected to the **View Application** page, where they can track the status of their application at any time. On this page, they can also download the generated PDF document of their application.

- **File Storage:** All documents, including the candidate's proof files and the final generated PDF, are securely stored on **Amazon S3**, ensuring safe and reliable storage with easy retrieval by authorized personnel.

C. Application Assignment Process - Admin

After a candidate submits their application, its status remains "pending" until an administrator assigns it to jurors for review. This process is managed through the **Admin Panel**, which is divided into three parts for efficient operation.

Part 1: Announcements Management

The first part of the admin panel allows the administrator to manage job announcements. This section includes:

- **List of Announcements:** All announcements posted by the admin are displayed here.
- **Modify Announcements:** The admin can modify details of each announcement.
- **Sorting Options:** Announcements can be sorted by:

- **Ongoing:** Announcements that are currently active.
- **Past Announcements:** Announcements that have already closed.
- **Upcoming Announcements:** Announcements that are not yet available on the applicant portal.

Part 2: Application Assignment

The second part of the admin panel shows applications that are awaiting assignment to jurors. It includes:

- **Pending Applications:** Displays five applications at a time that are waiting to be assigned.
- **"More Applications" Button:** Clicking this button leads to a new page listing all applications awaiting assignment.
- **Assign Jurors:** For each application, the admin can click the "Assign Jurors" button to assign a set number of jurors (between 3 and 7). Once the jurors are assigned, the application status is updated to "Processing."

Juror Assignment Algorithm

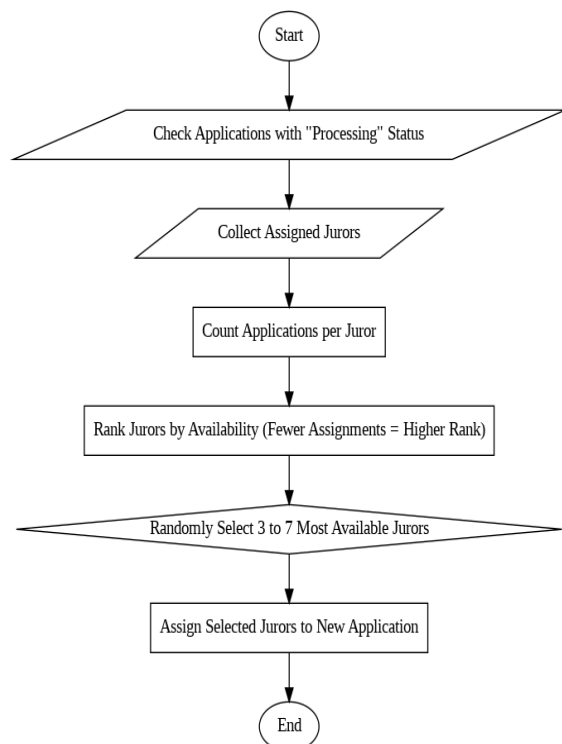
To ensure fairness and efficiency in the assignment process, the system uses a **random selection algorithm** based on jurors' availability.

No Manual Selection: Instead of allowing the admin to manually choose jurors by name or TC ID, the system randomly selects jurors based on their availability.

Transparency: This approach ensures that the assignment process is transparent, as the system operates like a "black box" and is not prone to favoritism.

Time Efficiency: By automating the selection process, the system saves time, especially when handling large numbers of applications. Each application is assigned 3 to 7 jurors, ensuring that no juror is overwhelmed by too many applications.

The algorithm works as follows:



Availability Check: The system first checks all applications with a "Processing" status and collects the jurors assigned to these applications.

Ranking Jurors: It then ranks all jurors based on the number of applications they have been assigned. The jurors with fewer assigned applications (more available) are ranked higher.

Random Selection: From this list, the algorithm selects the most available jurors (between 3 and 7) to be assigned to the current application.

Part 3: Application Evaluations

The third part of the admin panel displays evaluations of applications provided by jurors. This section allows the admin to view and manage jurors' feedback on each application. Further details on this will be discussed later.

D. Evaluation Process - Jurors

Once jurors are assigned to applications, they can access the application details to review the submitted documents, analyze the information, and make an evaluation.

Juror Panel

The juror panel displays all applications assigned to the juror, including their status:

- **Approved:** For applications that have been approved in the past.
- **Rejected:** For applications that have been rejected in the past.
- **Processing:** For applications that are still being evaluated.

The juror can click on the application they wish to evaluate. This action redirects them to the **Application Details** page, where they can review the applicant's personal information as well as the documents submitted during the application process.

Buttons for Evaluation

On the **Application Details** page, there are two main buttons that allow the juror to either view or add an evaluation:

- **"View Report":** This button is enabled if the juror has previously submitted an evaluation. Clicking it takes the juror to a new page where they can see the evaluation report, including the decision (approved or rejected), any messages, and the uploaded report file.
- **"Add Report":** If the juror has not yet made an evaluation, this button is enabled, allowing them to submit their judgment. The "View Report" button is disabled while this option is active.

When the **"Add Report"** button is clicked, the juror is presented with a form to enter their evaluation:

- **Approval Status:** A dropdown menu that allows the juror to choose between "APPROVED" and "REJECTED."
- **File Upload:** A file input to upload the evaluation report document.
- **Text Area:** A text area where the juror can provide their personal judgment or additional comments about the application.

Once the juror submits their evaluation, the report is stored, and both the juror and admin can access it (Read-Only).

E. Final Decision - Admin Panel

The third section of the admin panel contains all applications that have been assigned to jurors for evaluation. Once a juror submits their evaluation for an application, the status of the application is updated to reflect the juror's decision.

Application Status

- If a juror has not yet submitted an evaluation, the application remains unclickable (disabled) for them.
- If a juror has submitted their evaluation, the application status button becomes clickable. The color of the button changes:
 - **Red:** If the juror has rejected the application.
 - **Green:** If the juror has accepted the application.

Clicking on the application's status button directs the admin to the **Evaluation Details** page, where they can view the juror's decision, message, and the uploaded report file.

More Evaluations Button

In this section, there is also a "**More Evaluations**" button that, when clicked, redirects the admin to a new page listing all applications that have been assigned for evaluation. This allows the admin to easily navigate and manage multiple applications at once.

Final Decision

Once all the evaluations have been submitted by the jurors for an application, the admin is responsible for making the final decision. To handle this, the admin uses a form that includes:

- **Approval Status:** A dropdown menu that allows the admin to select either "**APPROVED**" or "**REJECTED**."
- **Text Area:** A space for the admin to enter their final judgment and comments regarding the application.

Once the final decision is made, it is recorded and the process is complete, and the application is updated with the status provided by the admin (who acts as a final jury). The applicant is notified and can check its application status through its portal.

5. Notifications & Status Updates

The system incorporates various methods for notifying users about their application status, ensuring that all involved parties are promptly informed of important updates. The notifications are sent through multiple channels:

App Built-In Notifications:

Notifications are stored within the database and can be displayed on each user's portal, regardless of their role. This allows users to see their updates directly within the app.

Twilio (SMS Notifications):

Twilio is integrated into the app for SMS notifications. However, it is not yet operable due to the need for a U.S. number to conduct tests. Alternatively, a Turkish number can be purchased for testing and operational use.

Brevo (Email Notifications):

Brevo is fully integrated and operational. It is used to send email notifications to users, ensuring that they are notified about important events related to their application process.

When Are Notifications Sent?

Notifications are triggered at various stages of the application and evaluation process to keep all parties informed. These include:

- Application Submission:

When a user submits a new application, a notification is sent to the admin who created the announcement, informing them of the new submission.

- Announcement Deadline Reached:

When an announcement reaches its deadline, the system sends an automatic notification to the admin who created the announcement, along with the total number of applications that were submitted for that announcement.

- Jury Assignment:

When jurors are assigned to an application for evaluation, a notification is sent to the assigned jury members, informing them of their new responsibility.

- Jury Evaluation Submission:

After a juror submits their evaluation, a notification is sent to the admin who created the announcement, informing them that the evaluation has been completed.

- Final Decision:

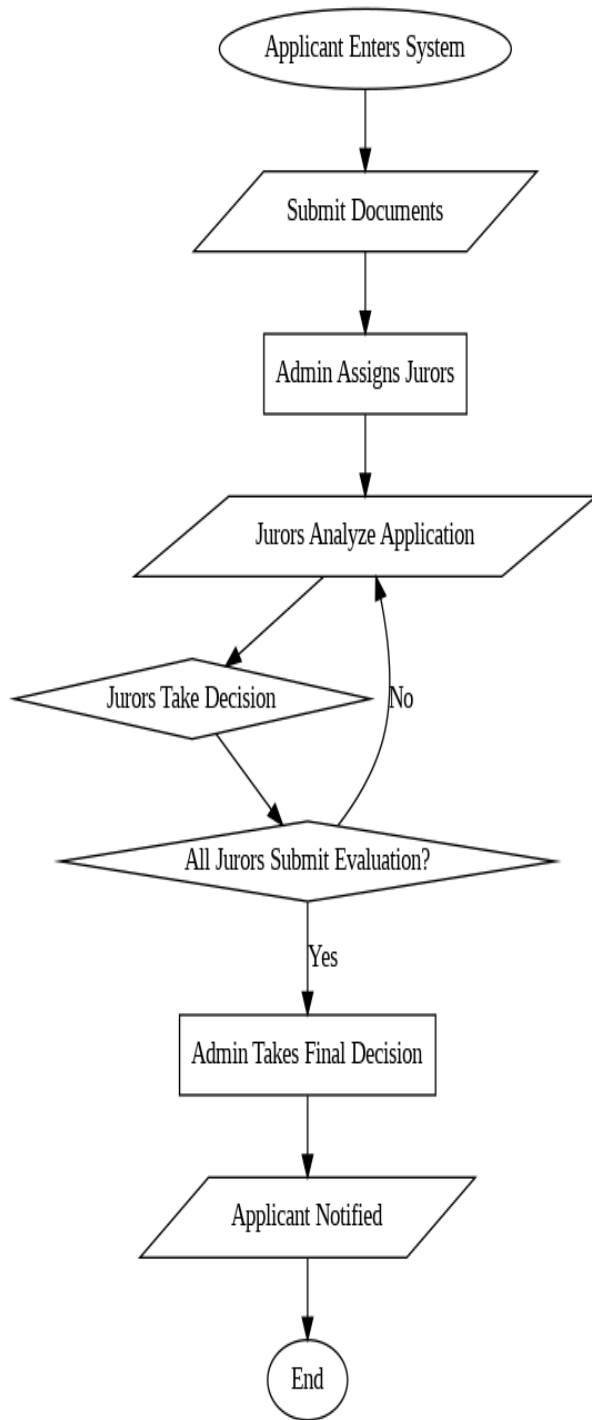
Once the final decision has been made by the admin, the applicant receives a notification informing them of the outcome of their application.

These notifications ensure transparency and timely communication throughout the application process.

Final Decision & Recruitment

Approved candidates are notified, and their details are stored in the system for further processing.

So from the submission of the application to the result, the system can be summarized as :



Manager Panel

The Manager acts as the backbone of the system. Unlike Admins and Jurors, the Manager does not intervene in the application process. However, all users—Admins, Applicants, and Jurors—rely on the criteria previously defined by the Manager. This makes the Manager a core component of the system.

The Manager Panel is divided into six sections, each serving a crucial role in structuring the application process.

i. Edit Criteria

This section presents a 3-in-1 form that allows the Manager to define the core evaluation criteria based on **KOÜ Atama Yönetmeliği**. It includes:

- Table 3 (Activity Categories & Activities):

A single form is used to add an entire category along with all its subsequent activities at once.

Example: *The L (GÜZEL SANATLAR FAALİYETLERİ) category contains 105 activities, all of which can be added in a single backend request instead of 105 separate calls. This ensures efficiency and reduces server load.*

- Table 1 (Minimum Activities):

Defines the minimum required activities for each faculty and position (e.g., Assistant Professor, Associate Professor, etc.).

- Table 2 (Minimum & Maximum Points):

Specifies the minimum and maximum points required based on faculty and position.

ii. Activities (table 3 - view)

Displays all activities currently available in the system along with their associated points.

Read-only access for everyone except the Manager.

The Manager has the ability to:

- Modify an existing activity.
- Delete an activity.
- Modify/Delete entire categories.

iii. Minimum Activities

Displays Table 1: The minimum required activities for each category, based on faculty and position.

Data entered via the Edit Criteria form is structured as:

- Ranges (e.g., A1–A4).
- Specific criteria (e.g., "Main Author").

Read-only for all users except the Manager.

The Manager can:

- Modify an existing criterion.
- Delete a criterion.

iv. Minimum & Maximum Points

Displays Table 2, showing:

- Minimum points required per category.
- Maximum points allowed per category.

Read-only for all users except the Manager.

The Manager can:

- Modify an existing point range.
- Delete a point range.

v. Case & Coefficient (Tablo 4)

According to **KOÜ Atama Yönetmeliği**, some categories (A, B, C, G) require either:

- A special case ("DURUM"), or
- A coefficient ("ÇARPAN") if there is no special case.

This section corresponds to: **Tablo 4 (Coefficients Table)**.

"Özel Durumlar" section (Page 22 of **KOÜ Atama Yönetmeliği**).

Used for automated point calculation in applications.

Only the Manager can modify, add, or delete data in this section.

vi. Manager Form (Dynamic Application Form)

One of the most critical sections of the system.

Represents the dynamic form that Applicants fill out during the application process.

The entire form structure is stored in the database.

Why store forms dynamically?

Ensures scalability as new categories or changes in **KOÜ Atama Yönetmeliği** can be accommodated without modifying the core system.

Only the Manager can:

- Modify an existing form.
- Delete an entire form.
- Add a new form.
- Modify/Delete specific fields in a form.

Each form corresponds to a category in Tablo 3 of **KOÜ Atama Yönetmeliği**, ensuring a structured and consistent evaluation process.

The Manager Panel serves as the core administrative interface, defining the rules, criteria, and scoring system that guide the application process. Unlike the Admin (who manages applications) or Jurors (who evaluate them), the Manager's role is to configure the evaluation framework that ensures consistency, fairness, and compliance with **KOÜ Atama Yönetmeliği**.

This structured approach provides flexibility and scalability, allowing the system to adapt to future changes without significant modifications.

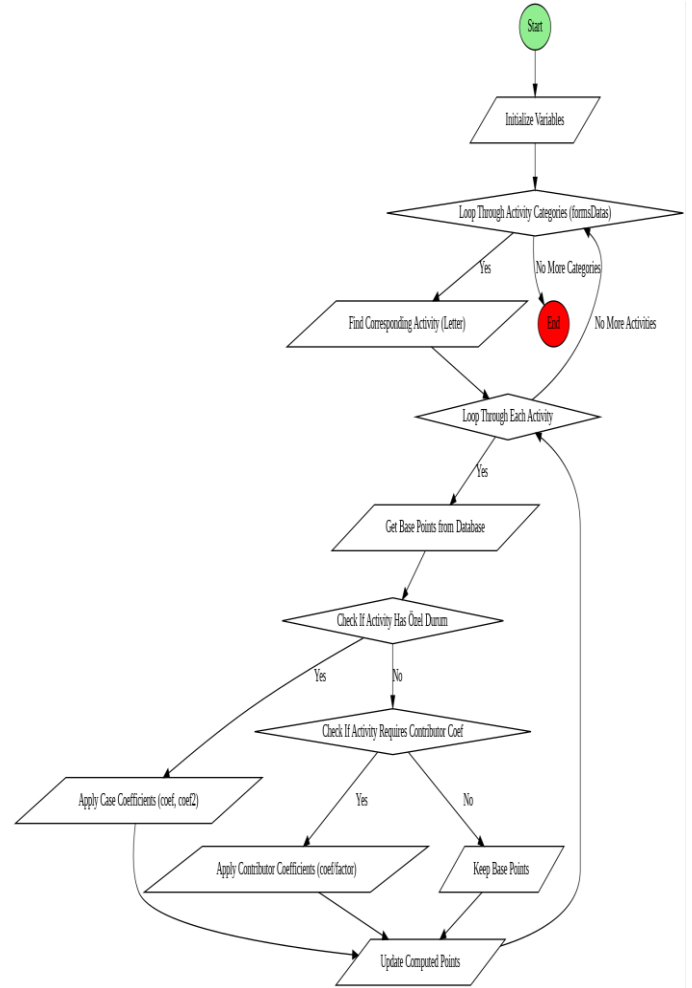
6. Point Calculation Algorithm and Mechanism

Point Calculation Function (computesPoints)

The computesPoints function is responsible for calculating the total points for each activity submitted by the applicant. The calculation follows the rules defined in **KOÜ Atama**

Yönetmeliği, using predefined coefficients and cases, which were entered by the Manager in the Case Coef Page.

Algorithm FlowChart



The function calls updatePoints(letter, number, round(activityPoints)) to store the final computed points for the activity.

Role of the Manager in the Calculation Process

- Defining Coefficients & Special Cases:
- The Manager predefines the coefficient values (coef, coef2) for **Özel Durumlar** in Case Coef Page.
 - If a special case exists, the calculation uses coef and coef2.
 - If no special case exists, a coefficient based on the number of contributors is used instead.
- Ensuring Fair & Transparent Evaluation: The use of predefined rules ensures that every applicant is evaluated consistently.

All criteria are stored in the database and dynamically applied during the point calculation process.

III. SYSTEM COMPONENTS

This section provides a detailed breakdown of the key components of the **Academic Staff Recruitment System**, covering the **frontend, backend, database, authentication, file storage, notifications, and external API integrations**.

1. Frontend (React.js & Bootstrap)

The frontend of the system is built using **React.js**, a modern JavaScript library that enables dynamic and responsive user interfaces. **Bootstrap** is used for styling to ensure a professional and mobile-friendly design.

Key Features:

- **User Authentication:** Login and registration with role-based access.
- **Dashboard:** Personalized interfaces for applicants, administrators, jury, and manager members.
- **Application Management:** Candidates can submit and track applications.
- **Real-time Updates:** Application status updates using API responses.

Main Technologies Used:

- **React.js:** Component-based architecture for efficient UI development.
- **React-native-dom Router:** Handles navigation between different views.
- **Bootstrap:** Ensures a responsive and clean user interface.
- **Fetch API:** Handles API calls to the backend without third-party libraries.

2. Backend (Node.js & Express.js)

The backend is developed using **Node.js** with the **Express.js** framework. It serves as the **central hub** that handles business logic, processes requests, and interacts with the database.

Key Features:

- **RESTful API Development:** Exposes endpoints for frontend interaction.
- **Role-based Access Control:** Manages different user permissions (Applicant, Administrator, Jury, Manager).
- **Validation & Error Handling:** Ensures data integrity and security. Displays clear message when an error occurs.

Main Technologies Used:

- **Node.js:** Asynchronous and efficient backend processing.
- **Express.js:** Lightweight framework for building APIs.
- **Mongoose:** ORM for MongoDB integration.
- **Multer:** File management.
- **Amazon S3:** File storage

3. Middle Backend-End Layer : React Contexts

In this project, an intermediate layer was introduced between the frontend and backend to efficiently **manage user sessions, role-based access, and API(backend) communication**. This layer is implemented using **React Contexts**, providing global state management and centralized logic for interacting with backend endpoints through **fetch API calls**. The following five main contexts were created:

- **UserContext:** Manages the authenticated user's session, including JWT token handling, personal data, and assigned roles. It's also here data minimum data are loading for each user role at the login, registration of the application.
- **ManagerContext:** For all operation related to manager (as talked before)
- **AdminContext:** For all operation related to admin (as talked before)
- **JuryContext:** For all operation related to jury (as talked before).
- **NotificationContext:** Handles all system notifications (success, error, info), ensuring consistent and user-friendly feedback throughout the application.

Although implemented within the frontend, these contexts serve as a "**middle backend-end**"^[1] layer, as they centralize logic that directly communicates with the backend while maintaining clean separation from purely presentational components.

Benefits of Context API

The use of the **React Context API** provides several advantages:

- **Native ReactJS API :** no installation required
- **Global State Sharing:** Data such as user roles or login tokens is accessible across all components without prop drilling.
- **Modular Logic:** API functions and role-based logic are encapsulated within each context, promoting code modularity and reusability.
- **Cleaner Component Structure:** Components remain focused on UI concerns while business logic resides in their respective contexts.
- **Improved Maintainability:** Separation of responsibilities between UI, logic, and data fetching simplifies debugging and future development.

Context API vs. Redux

While the Context API was sufficient for the project's scale and needs, integration with **Redux** could be considered in the future for more advanced scenarios involving:

- **Complex State Dependencies:** Redux's reducer-based pattern is more suitable for managing deeply nested or interdependent state updates.
- **Time-Travel Debugging:** Redux DevTools allow precise tracking of state changes, aiding in debugging complex workflows.
- **Middleware Support:** Redux middleware (e.g., `redux-thunk` or `redux-saga`) provides powerful control over

asynchronous logic, which could enhance the backend interaction layer.

4. Database (MongoDB)

The system uses **MongoDB**, a NoSQL database, for storing application-related data in a flexible JSON-like format.

Database Structure:

- **Models** : User, Faculty, Activity, Announcement, Application, Case, Coef, Department, Evaluation, FacultyGroup, Form, MinActivity, MinPoint, Notification.
- **Graphic Diagram** : The database diagram is given in a next section.

Why MongoDB?

- **NoSQL** : With the mongoDB noSQL structure, it's easy to handle data because of it's similarity with JSON.
- **Schema flexibility** allows dynamic data storage.
- **High scalability** supports growing user data.
- **Fast querying** for real-time application tracking.

5. Authentication & Security (JWT - JSON Web Token)

Authentication is implemented using **JWT**, ensuring secure user access and role management.

Authentication Process:

Users register with email and password.

The system verifies credentials and generates a **JWT token**.

The token is sent with each request for secure API access.

Middleware verifies tokens to allow access based on roles.

Security Measures:

Password hashing using **bcrypt**.

Token expiration to prevent unauthorized access.

Role-based permissions for different user types.

6. File Storage (Amazon S3)

The system integrates **AWS S3** for secure and scalable file storage.

Document Handling:

Applicants upload necessary documents (CVs, certificates, etc.).

Files are stored securely in **Amazon S3 buckets**.

URLs are generated for controlled access.

Also the table 5 pdf generated based on user inputs are sent to the bucket together with other files.

Benefits of Amazon S3:

Scalability: Handles large document storage efficiently.

Security: Provides encrypted file storage.

Accessibility: Fast retrieval and controlled access to files.

7. API Integrations (e-Devlet & Nüfus Müdürlüğü)

The system integrates with **e-Devlet & Nüfus Müdürlüğü APIs** to verify applicant information and documents.

Integration Steps:

The system sends user details to the **e-Devlet API** for identity validation.

The API responds with verification results.

Approved applicants proceed to the next recruitment stage (registration).

Why API Integration is Important?

Authenticity: Ensures that only legitimate candidates apply.

Automation: Reduces manual document checks.

Efficiency: Speeds up the verification process.

IV. FUNCTIONALITIES & USER ROLES

This section describes the **core functionalities** of the Academic Staff Recruitment System and details the different **user roles** involved in the platform. The system is designed to provide **role-based access control**, ensuring that each user has the necessary permissions to perform specific actions.

A. Core Functionalities

The system provides the following key functionalities:

1. User Authentication & Authorization

Users can register and log in using **JWT-based authentication**.

Role-based access ensures that different users have distinct permissions.

2. Application Submission & Tracking

Applicants can **fill out and submit** their applications online.

They can upload required documents, which are stored in **Amazon S3**.

A dashboard allows applicants to **track the status** of their applications.

3. Document Verification & Validation

The system verifies user details through the **e-Devlet & Nüfus Müdürlüğü API**.

The verification process ensures that only valid applications are processed.

4. Review & Evaluation by Jury Members

Jury members can access submitted applications for evaluation.

They can **approve, reject, or request additional documents**.

The review process is tracked, and evaluations are stored in the database.

5. Notification System (Twilio API)

SMS and email notifications inform applicants about application status updates.

Jury members receive alerts when applications require their attention.

6. Role-Based Dashboards

Each user sees a dashboard tailored to their role:

Applicants: Track application progress and receive notifications.

Administrators: Manage user accounts, oversee applications, and handle disputes.

Jury Members: Review applications and submit evaluations.

Managers: Oversee the entire recruitment process and generate reports.

B. User Roles & Permissions

The system defines **four main user roles**, each with specific permissions:

Role	Permissions
Applicant	- Register & log in – Create and update announcement - Submit applications - Upload documents - Track application status - Receive notifications
Administrator	- Manage user accounts - Assign jury members - Oversee application processing – Take the final decision
Jury Member	- Review & evaluate applications - Approve or reject an application - Leave feedback for applicants
Manager	- Generate reports & analytics - Manage system-wide settings

V. IMPLEMENTATION DETAILS

This section outlines the technical implementation of the **Academic Staff Recruitment System**, focusing on the **frontend, backend, database, authentication, storage, notifications, and API integrations**.

1. Frontend Implementation

The frontend is built with **React.js** and uses **Bootstrap** for styling. It follows a **component-based architecture**, ensuring modularity and reusability.

Key Implementation Details:

Routing: Implemented using **React-native-dom Router** for navigation.

Dynamic MultiStep form for user application

State Management: React's **useState** and **useEffect** hooks are used for handling state changes.

API Calls: **Fetch API** is used for communication with the backend. All of the API calls are centralised into Context using ContextAPI (UserContext, ManagerContext, AdminContext, JuryContext, NotificationContext)

Example API Call using Fetch – UserContext.js:

```
fetch('https://{api_address}/api/datas/applications/view', {
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json'
  }
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error('Error:', error));
```

2. Backend Implementation

The backend is developed using **Node.js** with **Express.js** to handle requests, process business logic, and manage the database.

Key Implementation Details:

Routing: Express.js is used to define RESTful API routes.

Middleware: JSON Web Token (**JWT**) authentication, multer with Amazon Web S3 are implemented.

Database Communication: **Mongoose** is used for MongoDB interactions.

External Notification Services : Twilio, Brevo

Node JS Utilities : Corn in inethrated to automatically send a notification to the Admin when an announcement expires

Example index.js Route:

```
const express = require('express');
const router = express.Router();
const auth = require('../middlewares/auth');

const announcementRoutes =
require('./announcementRoutes');
const applicationRoutes = require('./applicationRoutes');
/*other routes*/
```

```
module.exports = router;
```

Example applicationRoutes.js Route:

```
const express = require("express");
const router = express.Router();
const applicationController =
require("../controllers/applicationController");
const { applicationsDocUpload } =
require("../middlewares/awsMiddleware");

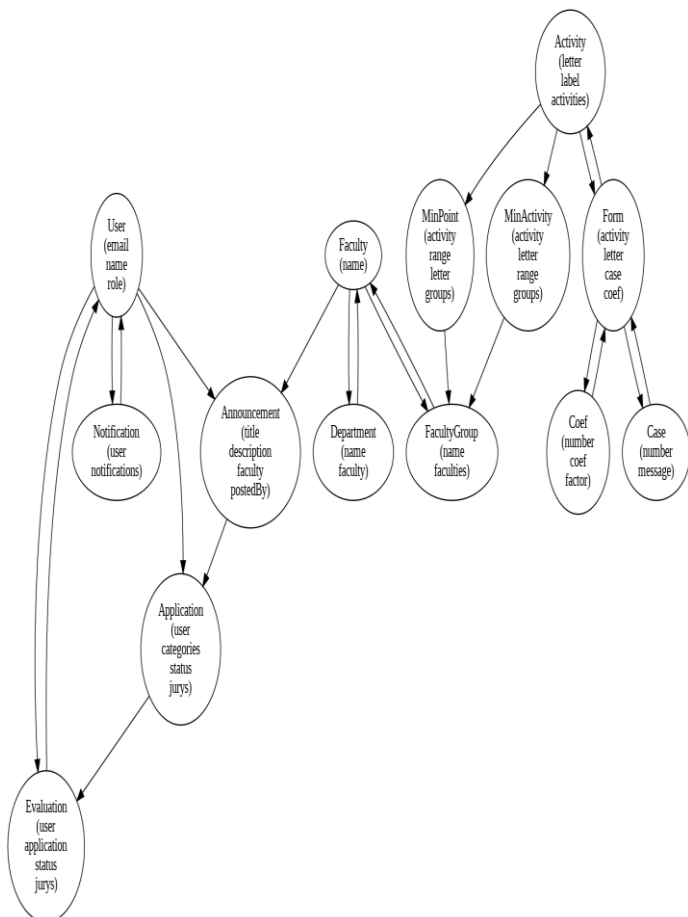
router.get("/get/all", applicationController.getApplications);
router.get("/user/get/:userId",
applicationController.getUserApplications);

module.exports = router;
```

3. Database Implementation (MongoDB)

The system uses **MongoDB**, a NoSQL database, to store user and application data.

Database Diagram:



Database Schema Examples:

User Schema:

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;
const uniqueValidator = require('mongoose-unique-validator');
const db = mongoose.connection.useDb("kouap");
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  surname: { type: String, required: false, trim: true },
  tcID: { type: String, required: true, trim: true },
  birthDate: { type: Date, required: true, trim: true },

  username: { type: String, required: false, trim: true },

  email: { type: String, required: true, unique: true, lowercase:
true, trim: true },
  phoneNumber: { type: String, required: true, trim: true },
  address: { type: String, required: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'admin', 'jury', 'manager',
'dev'], default: 'user' },
  cv : { type: String, required: function(){ return
this.role=='user' } },
  signature : { type: String, required: function(){ return
this.role=='user' } },

userSchema.plugin(uniqueValidator);

module.exports = db.model("User", userSchema);
```

4. Authentication (JWT - JSON Web Token) Seen from the Backend

User authentication is managed using **JWT**, which securely verifies and maintains user sessions.

Authentication Flow:

The user logs in and provides credentials.

The system validates credentials and generates a **JWT token**.

The token is used for secure API access.

Middleware ensures only authorized users can access specific endpoints.

Example Login Implementation:

```
exports.loginUser = async (req, res, next) => {
  try
  {
    let validePassword=false;
```

```

const { tcID, password } = req.body

const user = await User.findOne({ tcID: tcID })

if (!user)
{
  console.log('auth/user-not-found')
  return res.status(401).json({ error: 'auth/user-not-found' });
}

if (!isBcryptHash(password))
{
  validePassword = await
bcrypt.compare(password.toString(), user.password)
}
else
{
  validePassword = password.toString() ===
user.password.toString()
}

if (!validePassword) {
  console.log('auth/incorrect-password')
  return res.status(401).json({ error: 'auth/incorrect-password' });
}
else
{
  const token = generateToken(user._id);
  res.status(200).json({ message: "success", data: {
token: token, user : user } });
}

}
catch(error)
{
  console.log(error)
  res.status(500).json({ error });
}
};

```

5. File Storage (Amazon S3)

The system uses **Amazon S3** for **secure file uploads** and document storage.

File Upload Workflow:

The user uploads a document.

The file is stored in an **S3 bucket**.

The system generates a secure **URL** for access.

Example S3 File Upload using Multer:

```

const multer = require("multer");
const { S3Client } = require("@aws-sdk/client-s3");
const multerS3 = require("multer-s3");

const s3Client = new S3Client({
  region: process.env.AWS_REGION,
  credentials: { accessKeyId:
process.env.AWS_ACCESS_KEY_ID, secretAccessKey:
process.env.AWS_SECRET_ACCESS_KEY },
});

const MIME_TYPES = {
  "image/jpg": "jpg", "image/jpeg": "jpeg", "image/png":
"png", "application/pdf": "pdf",
  "application/msword": "doc",
  "application/vnd.openxmlformats-
officedocument.wordprocessingml.document": "docx"
};

const fileFilter = (req, file, cb) =>
cb(MIME_TYPES[file.mimetype] ? null : new Error("Type de
fichier non autorisé"), !!MIME_TYPES[file.mimetype]);

const multerS3Config = (folder) => multerS3({
  s3: s3Client, bucket:
process.env.AWS_S3_BUCKET_NAME, metadata: (req, file,
cb) => cb(null, { fieldName: file.fieldname }),
  key: (req, file, cb) => cb(null, `${folder}/${Date.now()}-
${file.originalname}`)
});

module.exports = {
  cvUpload: multer({ storage: multerS3Config("cv"), fileFilter
}),
  evaluationDocsUpload: multer({ storage:
multerS3Config("evaluationReports"), fileFilter }),
  applicationsDocUpload: multer({ storage:
multerS3Config("applications"), fileFilter }),
};

```

6. Notification System (Twilio API)

Twilio is integrated to send **SMS and email notifications** regarding application status updates.

Example Twilio Integration:

```

require('dotenv').config({ path: '../shared/.env' });
const mongoose = require('mongoose');

```

```

const Notification = require('../models/notificationModel');
const User = require('../models/userModel');

const twilio = require('twilio');

const client = new
twilio(process.env.TWILIO_ACCOUNT_SID,
process.env.TWILIO_AUTH_TOKEN);

exports.sendSms = async (to, message) => {
  try {
    const response = await client.messages.create({
      body: message,
      from: process.env.TWILIO_PHONE_NUMBER,
      to: to,
    });
    console.log("SMS envoyé avec succès :", response.sid);
    return response;
  } catch (error) {
    console.error("Erreur lors de l'envoi du SMS :",
error.message);
    throw error;
  }
};

```

7. API Integrations (e-Devlet & Nüfus Müdürlüğü)

The system integrates with **e-Devlet & Nüfus Müdürlüğü APIs** for applicant verification.

Integration Process:

The system sends applicant data to **e-Devlet API**.

The API validates the data and responds with verification results.

The system updates the application status accordingly.

Example API Request:

```

const generateSoapRequest = (tcID, name, surname,
birthYear) => `
<?xml version="1.0" encoding="utf-8"?>
<soap12:Envelope
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap12="http://www.w3.org/2003/05/soap-envelope">
  <soap12:Body>
    <TCKimlikNoDogrula
xmlns="http://tckimlik.nvi.gov.tr/WS">
      <TCKimlikNo>${BigInt(tcID)}</TCKimlikNo>
      <Ad>${name}</Ad>
      <Soyad>${surname}</Soyad>
      <DogumYili>${birthYear}</DogumYili>

```

```

    </TCKimlikNoDogrula>
  </soap12:Body>
</soap12:Envelope>`;

exports.controlUser = async (req, res) => {
  const { tcID, name, surname, birthYear } = req.body;
  const userID = await User.findOne({ tcID });

  if (userID) return res.status(401).json({ error: 'auth/user-
already-exists' });

  const soapRequest = generateSoapRequest(tcID, name,
surname, birthYear);

  try {
    const response = await
fetch('https://tckimlik.nvi.gov.tr/Service/KPSPublic.asmx', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/soap+xml; charset=utf-8',
        'SOAPAction':
'http://tckimlik.nvi.gov.tr/WS/TCKimlikNoDogrula',
      },
      body: soapRequest,
    });

    if (!response.ok) return res.status(response.status).json({
error: 'Failed to call SOAP API' });

    const responseText = await response.text();
    const { "soap:Envelope": { "soap:Body": [{
TCKimlikNoDogrulaResponse: [{
TCKimlikNoDogrulaResult: [result] }] } } } = await
xml2js.parseStringPromise(responseText);

    if (result === 'true') return res.status(200).json({ message:
'User information is correct' });
    return res.status(400).json({ error: 'User information is
incorrect' });

  } catch (error) {
    console.error('Error calling SOAP API:', error);
    return res.status(500).json({ error: 'Failed to call SOAP API'
});
  }
};

```

Twilio Integration and Geographical Limitations

The free version of Twilio allows sending messages only to phone numbers based in the USA. Due to this geographical restriction, I had to temporarily switch to **Brevo** (formerly Sendinblue) for email notifications. Fortunately, the system already has its own notification system, utilizing its database for alerts.

Despite this limitation, Twilio is well-integrated into the system and functions smoothly. The only issue encountered was the inability to purchase a Turkish phone number, which is restricted in the free version of Twilio.

Twilio API Documentation. <https://www.twilio.com/docs>.

Brevo API Documentation <https://developers.brevo.com/>

Amazon S3 Documentation
<https://docs.aws.amazon.com/s3/>

Tools and Libraries

Express.js Documentation. <https://expressjs.com>.

MongoDB Documentation.
<https://www.mongodb.com/docs/>

VI. LEXICON

[1] Middle Backend:

A custom architectural term used in this project to refer to a layer between the frontend and the backend. This "middle backend" is implemented using React Contexts and is responsible for managing global application state, handling API calls to the backend, and controlling user roles and logic flows. Although technically part of the frontend, this layer performs backend-related operations (such as fetching data, processing responses, and managing authentication tokens), thus acting as a hybrid interface between UI components and backend services. It improves code organization, modularity, and ensures a clean separation between presentation and logic layers.

VII. Conclusion & Future Work

This section summarizes the achievements of the project, highlights its strengths, and outlines potential areas for future improvements.

1. Summary of Project Achievements

The project successfully implemented an academic application system that allows candidates to submit applications, receive notifications, and be evaluated by a jury. The integration of e-Devlet and Nüfus Müdürlüğü APIs streamlined candidate verification, while AWS S3 ensured reliable document storage.”

2. Areas for Improvement and Future Enhancements

Potential Improvements: Future enhancements could include adding machine learning models to assist juries in evaluating candidates’ applications more efficiently. Additionally, a mobile application version could improve accessibility for users on the go.”

Long-Term Vision: In the long term, the system could be scaled to support multiple universities, allowing candidates to apply to different academic programs with a single account. Further integrations with other government agencies could improve the verification process.

VIII. REFERENCES

Books & Articles:

KOU, KOÜ Atama Yonetmelig’i

Documentation:

React.js Documentation. <https://reactjs.org/docs>.